

Handoff — 2026-07-27

Read this first if you are picking up ABRA in a new session.

The previous session ended mid-stride: a human was playing MAG on a live Showdown server, found a real bug by playing it, and the fix for that bug changed a feature's meaning. That is the state you are inheriting. Nothing is broken; two things are *stale*, and this document says exactly which.

o. DO THIS FIRST. THE WEIGHTS ARE STALE.

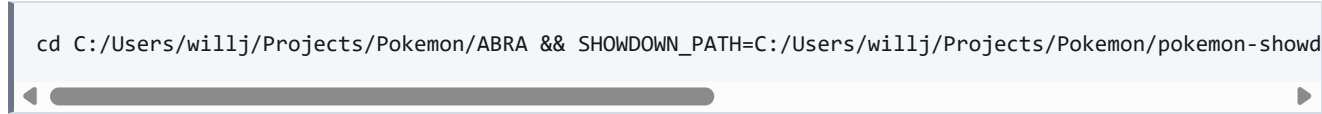
`engine/board.js` feature #47 `deadNoLastMove` changed meaning at the very end of the session (it is now gated on move order — see §2). The fitted weights predate the gate. The feature **count** still matches at 47, so nothing errors and nothing warns: MAG will load, run, and play with a weight fitted against a different definition of that feature. This is the silent-failure shape this project keeps getting bitten by, so it is item zero.

```
cd C:/Users/willj/Projects/Pokemon/ABRA && SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showd
```



Then the no-popularity variant, which is **also** stale and is additionally still at 46 features, so it cannot be loaded at all until this runs:

```
cd C:/Users/willj/Projects/Pokemon/ABRA && SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showd
```



The flags are environment variables, not CLI flags. `DROP=` and `OUT_WEIGHTS=`. A previous attempt used `--drop popularity --out ...`; `fit_policy.js` ignored both, quietly did a *plain* refit, and overwrote `data/policy-weights.json`. It printed `-> data\policy-weights.json` and exited o. The feature is called `priorLogP`, not `popularity`.

Each fit takes roughly 40 minutes on this machine.

The rule that this session learned the hard way

Never edit `board.js` while a fit or a self-play run is in flight. Node caches the module at require time, so a process started before the edit keeps the old feature vector and a process started after gets the new one — in the same output file, with no error. This ruined a 4,847-pair experiment (§3) and a 40-minute fit in a single evening. Finish the edit, refit, *then* run.

1. WHERE EVERYTHING IS

Thing	State
engine/board.js	47 features. <code>deadNoLastMove</code> is gated on <code>movesFirst</code> . Committed.
data/policy-weights.json	47 features, fitted against the ungated definition. Refit.
data/policy-weights-nopop.json	46 features. Unloadable. Refit with <code>DROP=priorLogP</code> .
data/h2h-nopop-greedy.jsonl	Contaminated, discard. See §3.
engine/mag_bot.js	Live-odds page at <code>localhost:8081</code> . Line graph removed at user request.
Showdown server	<code>localhost:8000</code> , bot logs in as MAGBOT

Two processes were running for the user when the session ended (the Showdown server and the bot). They are not managed by anything; if they are gone, restart them.

2. WHAT WAS FIXED: ENCORE, AND WHY THE FIRST FIX WAS NOT ENOUGH

The user, playing the bot, reported: *"this thing is encoring things that just switched in."* Encore fails outright against a target with no last move. MAG could not see it — the whole "this move cannot work right now" family (`deadStatus` , `deadSide` , `deadField` , `deadWeather` , `deadStall`) was missing this member.

Added `GAME_RULES.needsTargetToHaveMoved` = `['encore', 'disable', 'torment', 'spite', 'mimic', 'instruct', 'mirrormove']` and feature #47 `deadNoLastMove` . Encore appears on **5.34%** of teams in this format (Disable 0.80%, Instruct 0.31%), so it is not an edge case.

The fit agreed emphatically: `deadNoLastMove` = **-2.943**, 95% CI [-3.359, -2.528]. One of the strongest negatives in the model. Humans essentially never click it.

****Then the user added: "ESPECIALLY IF THEY HAVE PRANKSTER ENCORE" — and that broke the first fix.****

The flag is not "the target has no last move." It is "the target has no last move **and I resolve first.**" A fresh switch-in has no last move *at the moment of the decision*, but it is about to make one. If Encore is **slower**, the target moves, gains a last move, and Encore lands — that is the normal, correct way the move is used. Only a **faster** Encore hits a target that still has not moved.

Which makes Prankster the worst case rather than a footnote: +1 priority on a status move means Whimsicott, Sableye and Grimmsnarl essentially always resolve first, so Prankster Encore into a fresh switch-in fails **every time**. The ungated version flagged the good play (slow Encore) exactly as hard as the bad one.

The gate is `* x[FEATURE_INDEX.movesFirst]` , and it lives at the bottom of `featuresFor` with the rest of the dead-move family, because move order is not settled until Tailwind, Trick Room and priority have been applied further up. Verified:

Whimsicott (Prankster) -> fresh Garchomp	deadNoLastMove=1.00	movesFirst=1.00
Torkoal (slow, no Prankster) -> fresh Garchomp	deadNoLastMove=0.00	movesFirst=0.00
Whimsicott -> Garchomp that already moved	deadNoLastMove=0.00	movesFirst=1.00

Test script: `scratchpad/encore_test.js` . **Worth promoting into the repo as a real test** — this is the second time a feature in this family was wrong in a way no automated check caught, and both times a human playing the bot for ten minutes is what found it.

3. THE CONTAMINATED EXPERIMENT — DISCARD AND RE-RUN

The open question was the user's: *"would greedy mode still be the best if pop was gone?"* — the fourth cell of a 2×2 (popularity on/off × greedy/sampling). The run reported:

*no-popularity + greedy vs the random bot: **35.4% of decisive pairs**, 4,847 pairs*

This is not a finding and must not be reported as one. Losing two-to-one to a bot that clicks at random requires *confidently bad* play, not weak play. Cause, from file timestamps:

- `policy-weights-nopop.json` fitted at **46** features, 00:31
- `board.js` went to **47** features, 00:58
- the run was still writing at **01:05**

The run straddles the edit. Games before and after were not playing the same model. Re-run after both refits in §0, and **do not touch** `board.js` **while it runs**.

For context, the three cells that *are* clean, all paired, open sheets, switching off:

comparison	decisive pairs
greedy vs sampling, both full model, vs random bot	91.8% (raw 75.3%)
new MAG vs old MAG, 19,440 games	55.5% [53.8, 57.3]
full model vs random bot	81.9% (raw 66.3%)

Greedy instead of softmax sampling was worth **+9 points** — five times everything the feature work bought. Dropping `priorLogP` improved top-1 human-click prediction 34.6% → 35.2%.

4. NEW FINDING: THE SET SAMPLER IS WRONG IN BOTH DIRECTIONS

The user, watching a battle: *"we should talk about the need for two stab moves, it's probably not a realistic team construction choice for most pokemon. some will have two, but gunk and dire claw are not it."* The bot had brought a Sneasler with **Dire Claw and Gunk Shot** — both Poison.

Measured against open team sheets (all four moves public, no revelation bias) from **2,245 clean games**, 124 species. "Two or more *attacking* moves sharing a type":

	humans	generated	gap
all sets	28.5%	34.7%	+6.2
Incineroar	0.0%	22.5%	+22.5
Raichu	9.3%	30.0%	+20.7
Farigiraf	0.0%	14.5%	+14.5
Garchomp	26.7%	37.0%	+10.3
Kingambit	98.9%	71.0%	-27.9
Floette-Eternal	100.0%	83.0%	-17.0

The user's instinct was right and the data is sharper than the rule. Kingambit is the counter-example that matters: Sucker Punch and Kowtow Cleave are both Dark and virtually every Kingambit runs both, so a blanket "no two same-type attacks" rule would be wrong 99% of the time for that species. The sampler over-produces doubles for Incineroar and *under*-produces the mandatory ones for Kingambit.

Cause — `engine/champions_sim.js:142` → `set_priors.fillSet` draws each move **independently** from $P(\text{move} \mid \text{species})$. Real sets are correlated *within a role*: Gunk Shot is a perfectly normal Sneasler move (it appears on real sheets in the store) and so is Dire Claw, but they compete for one slot, so marginal sampling pairs them at roughly $P(a) \cdot P(b)$. Independence cannot represent a slot.

This matters more than it looks. Only ~ 1.38 of 4 moves are revealed per set on the ladder, so whatever fills the other 2.6 dominates any result built on generated teams — that is ADR-001's lesson, and it has now recurred in a subtler form.

Suggested fix, not yet built: sample *whole observed sets* (or nearest-neighbour on the revealed moves) instead of four independent marginals. Open sheets give $2,245 \text{ games} \times 12 \text{ complete sets}$ of ground truth to draw from, which is plenty. Audit script: `scratchpad/stab_audit.js` — promote it.

5. STANDING RULES — THESE ARE THE USER'S, NOT SUGGESTIONS

- **Never ask the user to run commands. Run them.**
 - **Always be pushing** — commit and push in the same pass as any change.
 - **Never** `git merge -X ours` **on this repo.** It has duplicated the store three times.
 - **GARBODOR:** never baseline against the raw ladder store. 87% of it is bots, forfeits and stubs. Everything behavioural goes through `engine/quality.js` (`isClean`, `loadGames`).
 - Measured, not asserted. No hand-typed constants. **Report negative results plainly.**
 - Docs + CHANGELOG + version in the same pass as code.
 - S12 no hardcodes, S13 no hand-maintained state.
 - **Explain in plain English.** The user is not a stats person — lead with meaning, not decimals.
 - **PDFs, not markdown**, for anything delivered. `build/build_pdfs.js` derives the list.
 - `bash build/publish.sh` is the only publisher.
 - `SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown` for anything running battles.
-

6. OPEN, IN ROUGH PRIORITY ORDER

1. **The two refits in §0**, then the clean re-run in §3.
 2. **Joint scoring is not wired into the player.** `jointFeaturesFor` and `data/policy-weights-joint.json` exist and are fitted; `magnemite.js` still scores each Pokémon's move independently. This is the "Helping Hand + Heat Wave double KO should score higher" work — built, measured, not connected.
 3. **Promote the two scratchpad scripts** (`encore_test.js` , `stab_audit.js`) into the repo.
 4. **Set sampling** — §4.
 5. **Volatiles are still invisible:** perish count, being Taunted / Encored / confused / behind a Substitute. The user asked directly whether a switch would fire under perish trap or a bad Encore. It will not — MAG cannot see any of it.
 6. **The defensive type chart as a graded feature.** MAG sees what *it* hits for $\times 4$; it does not symmetrically see that the thing in front of it hits *back* for $\times 2$.
 7. **Mega detection should read the revealed sheet item**, not the modal item.
 8. **CHOMP overhaul:** `xatu-context-sets.json` and `pokemon-roles.json` are both UNSAFE (built on the dirty store). Regenerate on clean data, re-run CHOMP-EV (currently 0.5129, CI [0.5021, 0.5395]), then test bringing MEW in.
 9. The site's MAGNEMITE room self-check fails — the browser re-implementation is still 21-feature.
 10. 14 files carry hardcoded Pokémon names per `engine/conformance.js` .
-

7. THE HONEST HEADLINE

MAG beats a random bot decisively and beats its own previous version by a real but modest margin. The single biggest measured improvement all session was **not** a feature — it was taking the best-scoring move instead of sampling from the distribution (+9 points). That is the imitation-ceiling result stated plainly: fitting to human clicks and winning games are different objectives, and this model is fitted to the first one.

The two bugs that mattered most this session were both found by a person playing the bot for ten minutes, not by any automated check. Keep doing that.